

CS496 Software Project: Karson Institute Digital Library

Vilnis Jatnieks, Aidan Marshall

2026-02-25

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Ricarda Easton
- Client title: Program Assistant
- Client email address: reaston@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: She was Vilnis's manager at the Karson Institute in Spring 2025.

2 Project Description

2.1 Overview

It's hard to find books in the Karson Institute's library. They have a constantly growing collection of books, but they lack a real organizational system. When a staff member wants to find a book, they spend a long time searching for it. Additionally, when the Karson acquires new books to join its collection, they have no easy way log it. My project, a web app that acts as a digital library, would solve both issues.

2.2 Key Features

- Staff can browse, search, and sort books by any field.
- Book fields include title, author, ISBN, LOC number, number of copies, and the location of the book in library (e.g. Shelf 2, Row 3), type, and tags.
- Categorize unpublished works. Is the work a collection of letters, unfinished writings, speeches, or a stand-alone work that never made it to publication? We need to clarify the types w/ the client and make this a field.
- Staff can give books tags such as the genre, subgenre, and topic.
- Staff with editor permission can edit, add or delete books from the library.
- Staff with admin permission can assign or revoke permissions from other staff.
- When the ISBN is entered, the system should autofill the rest of the fields. Then, the user confirms that the suggested fields are accurate.

- When the barcode of a book is scanned, the system should read the ISBN.
- Users can check out/check in books.
- Users can see recommendations when they view a book.
- Users will get a notification when the due date is approaching and after books are due

2.3 Why this Project is Interesting

This project is most interesting to me because it provides real value to the client. The student staff at the Karson Institute spend a lot of time doing tedious, manual labor. If this is automated, they can focus on more creative projects. Moreover, I'm excited to see what additional features I can add to the project such as a recommender system, barcode scanning, or LLM integration. Features will only be added after a fully functional product is presented to the client, but I think there are a lot of ways I could expand on the project to make it stand out.

2.4 Areas of CS required

- Databases
- Web Development
- Software Engineering

2.5 Potential Concerns and Questions

We are concerned with implementing too many features for this project. We want to make sure that the features that we do implement are valuable to the users.

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

The project is generally the same as the original proposal (Vilnis's client - Karson Institute). However, we added additional features in order to make our project viable. The first feature we added was a barcode scanner. This will help speed up book intake, because the Karson Institute will not have to enter all the book's information. We also decided to add email notifications when books are overdue. The Karson Institute has students who checked out books, but have not returned them. This feature hopes to encourage students to return these books. Finally, we would like to implement a book recommendation system where similar books are shown to users based off of the book they are currently viewing.

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR0	Restrict checkout	System	Users with overdue books cannot check out additional books
NFR1	Password strength	Security	Passwords must have 8+ characters with 1 capital letter, 1 number, and 1 special character
NFR2	SSO	Security	Allow users to use Loyola SSO
NFR3	Notifications	UI	Bell icon + number of notifications
NFR4	Limit Review Posters	System	Only users who have checked out a book can leave a review for that book
NFR5	Comment filter	System	Filter out profanities and unwanted language

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
A0	Users CRUD	8	As a/an Admin, I want to CRUD other users, so that I can assign users to have staff permissions and delete bot accounts
S1	Book CRUD	8	As a/an Staff, I want to CRUD books, so that I can maintain the collection of books
S2	Fill book info via ISBN	2	As a/an Staff, I want an alternative way to fill book information with an ISBN number, so that make logging a book easier
S3	Scan barcode	1	As a/an Staff, I want to add book by scanning barcode, so that make logging even easier
U4	Search for book	2	As a/an User, I want to search for a book, so that I can find what I want
S5	Check out/in book	5	As a/an Staff, I want to check out/in a book, so that I can update the current status of the collection
U6	Reserve book	2	As a/an User, I want to reserve book, so that I can let people know to not take it
U7	Reminder email	2	As a/an User, I want to receive reminder emails when due date is approaching, so that I can return my book on time
U8	View books	2	As a/an User, I want to view the books, so that I can find a book that I'm
U9	Filter the view of the books	1	As a/an User, I want to filter book, so that I can easily change view
U10	Recommended books	5	As a/an User, I want to see recommended books, so that I can find something that is interesting to me
S11	View checked out books	2	As a/an Staff, I want to see all books that have been checked out, so that I know which books are not currently in the library
U12	Sort the view of books	1	As a/an User, I want sort books by when they were published, so that I can see the newest published books
U13	Upvote/downvote comments	2	As a/an User, I want to react to comments, so that I can show my approval
S14	Tags CRUD	8	As a/an Staff, I want to CRUD tags for books and other works, so that I can organize my collection how I would like
S15	Late users	2	As a/an Staff, I want to view users who have overdue books, so that I have an easy way to follow up with students who need to return a book
U16	User view my history	2	As a/an User, I want to see previous books I checked out, so that I can remember what books I previously read

S17	Staff view book history	2	As a/an Staff, I want to see a book's history, so that I can see where a book has went
U18	Popular books	1	As a/an User, I want to see which books have been checked out the most, so that I can read what other books people find interesting
U19	Password reset	2	As a/an User, I want to reset password, so that I can log if I forget my password
U20	View book covers	2	As a/an User, I want to view book covers, so that I can see the cover artwork
U21	Rate book	3	As a/an User, I want to rate a book I read, so that I can let other people know my opinion on the book
U22	Contact staff	1	As a/an User, I want to send emails to the staff, so that I can ask questions about the collection
U23	Book preview	2	As a/an User, I want to preview the first pages of a book, so that I can read a snippet of the book before checking out
U24	Request extension	1	As a/an User, I want to request an extension of a book, so that I can keep the book for longer
S25	Approve extension	1	As a/an Staff, I want to approve an extension of a book, so that I can allow people to keep books for longer
U26	Create account	2	As a/an User, I want to create an account , so that I can log back in
U27	User get notifications	3	As a/an User, I want to get notifications about overdue books, new books added, receive notis from admin, etc., so that I know what actions need to be taken
A28	Admin send notifications	2	As a/an Admin, I want to push notifications to staff, so that I can send information
U29	Comments CRUD	8	As a/an User, I want to comment reviews on books, so that I can share my opinion about the book
U30	Report comments	1	As a/an User, I want to report comments, so that I can ensure a safe space
U31	Log in/log out	2	As a/an User, I want to log in and log out, so that I can access my account
U32	Popular books by genre	1	As a/an User, I want to see which books have been checked out the most in each genre, so that I can see what other people like
Total:		89	

Table 2: Functional requirements as User Stories.

3.2.1 AI User Story Analysis

Model: Claude Opus 4.5

Prompt: The attached .pptx is the project my partner and I am working on for our Senior Capstone, and the .xlsx are the current user stories that we have.

Are there any important/valuable user stories we are missing? If yes, what is it and explain why you believe it is needed.

Response: I'll review your capstone proposal and current user stories to identify any gaps.

Check the skill file for working with xlsx files

Read the user stories spreadsheet

Now let me extract the key information from your PowerPoint to understand the full project scope:

Read the capstone proposal slides to understand the project scope

Based on my analysis, here's the most important missing user story:

Missing User Story: View Book Availability/Location

As a User, I want to see whether a book is available and where it's located, so that I know if I can get it and where to find it on the shelves.

Why this is critical:

Your proposal slides explicitly frame the core problem: a student comes to the Karson Institute, searches through shelves, can't find a book, asks staff, and then discovers it's already checked out. Your current user stories have:

U4: Search for book

U8: View books

S11: Staff view checked out books

But no user-facing story that shows availability status or physical location. The student still has to ask staff or wander the shelves—which is the exact pain point your project is supposed to solve.

This story directly addresses the "Finding a Book is Hard" problem from slides 3-4 and should be Sprint 1 or 2, High priority since it's central to your value proposition.

Secondary gap worth noting:

You have S2 (ISBN lookup) and S3 (barcode scan) for adding books, but no story for bulk import of the existing 1600+ book collection. That's a real operational concern for launch—your client won't manually enter 1600 books. Consider whether that needs to be scoped.

Analysis: The first story that Claude proposed is a great idea that we overlooked. We did specifically mention that there would be a feature to find the location of a book. For the second user story, Claude is right that it is inconvenient to have to scan 1,600 books individually. However, we could not think of a reliable way to bulk import books accurately. The only solution to ensure that each book is imported is through a manual individual process. We will not include this story in our functional requirements.

4 System Design

4.1 Architecture

We will be using a layered architecture. The 3 layers of our application are the visualization layer "components" (where the reusable React components that are used throughout the pages live), the routing layer "app" (where all the pages and API routes live), and the data layer "lib" (establish a database connection). Additionally, we will use a RESTful architecture in order for our frontend to be able to communicate with our backend.

Some of the modules throughout our program are Users (admin, staff, or normal user), Works (will mostly be books, but could be speeches or other types of written works), Comments (written by users on posts), Notifications (sent to users for various reasons, can be sent by system or admin), Author (of a work), Tags (attached to works to organize collection).

4.2 Diagrams

4.3 Technology

Technology	Function	Justification
Next.js	Frontend framework	Easier routing
React	Frontend framework	Reusable components, hooks
Tailwind	CSS library	Easier CSS
ShadCN	Component library	Pre-made components
Node.js	JS Runtime	Use one programming language across frontend and backend
TypeScript	Programming language	Strongly typed, self-documenting
ESLint	Linters	Enforce code style
Jest	Test framework	Easier unit testing
Postman	Backend testing	Easier manual testing, backend development
PostgreSQL	Database	Relational database, speed

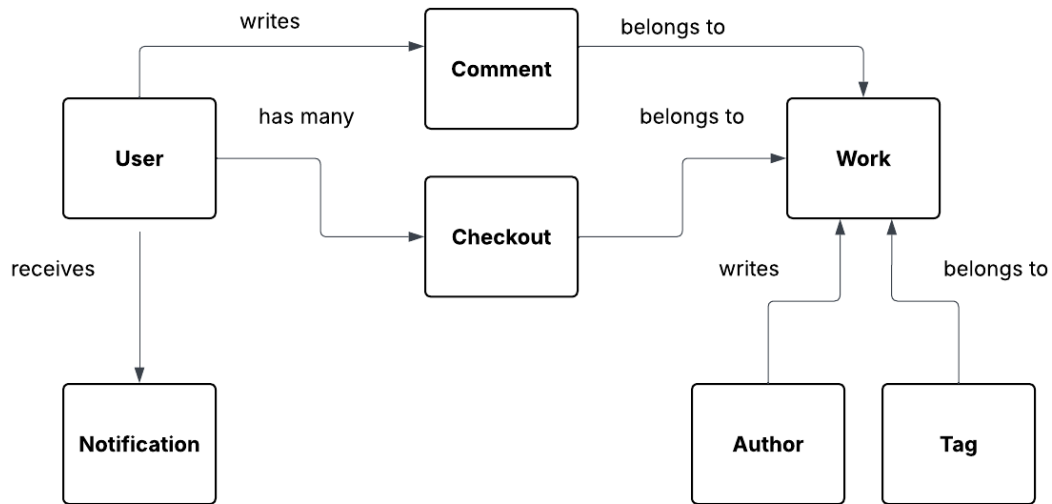


Figure 1: Simplified Class Diagram

4.4 Coding Standards

We will be following standard JavaScript naming conventions for this repository. We will use camelCase for variables, SCREAMING_SNAKE_CASE for constants, PascalCase for components, and kebab-case for files. Features need to be created on new branches. All new branches must be created off of the dev branch. The naming convention for branches will look like "<user-story-id>-<user-story-name>". An example of this branch naming convention is "A0-users-crud". Pull requests must be reviewed by each other before being merged to the dev branch. GitHub Actions will also be set up to automatically run test cases. A pull request cannot be merged unless all test cases have passed. We will merge to the dev branch during sprints, and merge to the main branch at the end of each sprint (hotfixes are an exception). We will enforce a minimum of 80% line coverage for our repository.

4.5 Data

Data will be stored in a relational database. The main structure of the data is defined by the Entity Relationship Diagram. We chose to use an SQL database over a NoSQL database because our data has clear relationships (like the relationships between work, checkout, and user).

4.6 UI Mocks

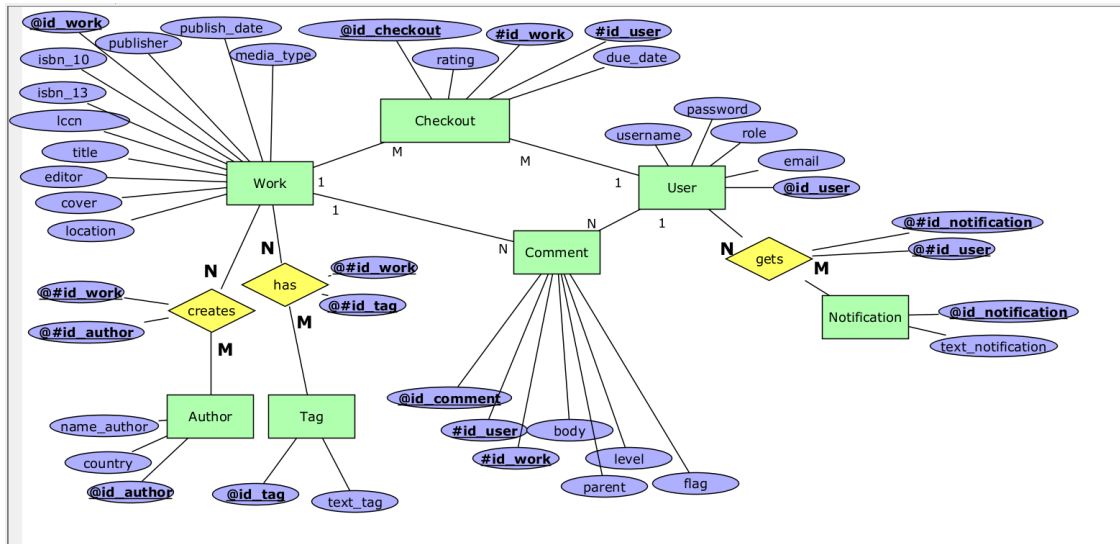


Figure 2: Entity Relationship Diagram

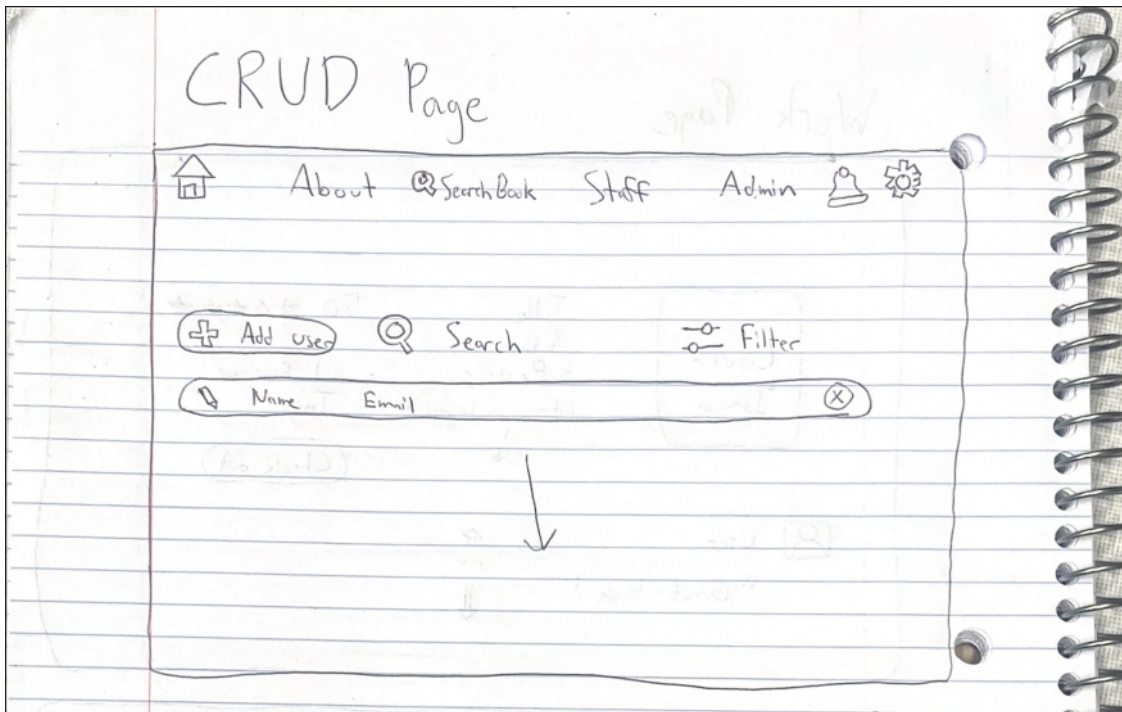


Figure 3: CRUD Mockup (Users and Works)

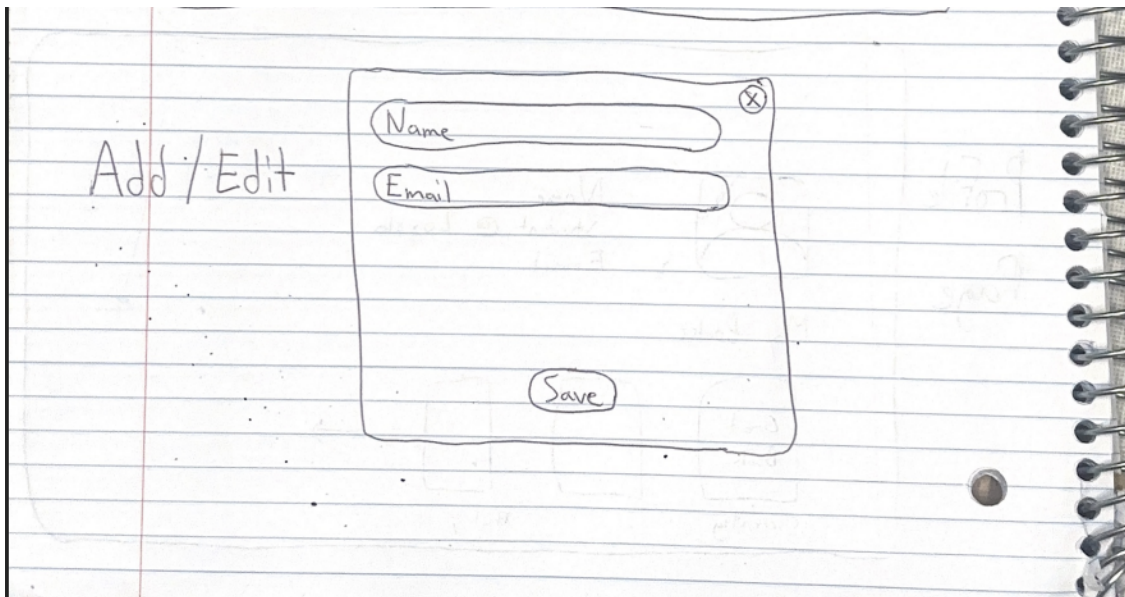


Figure 4: CRUD Modal

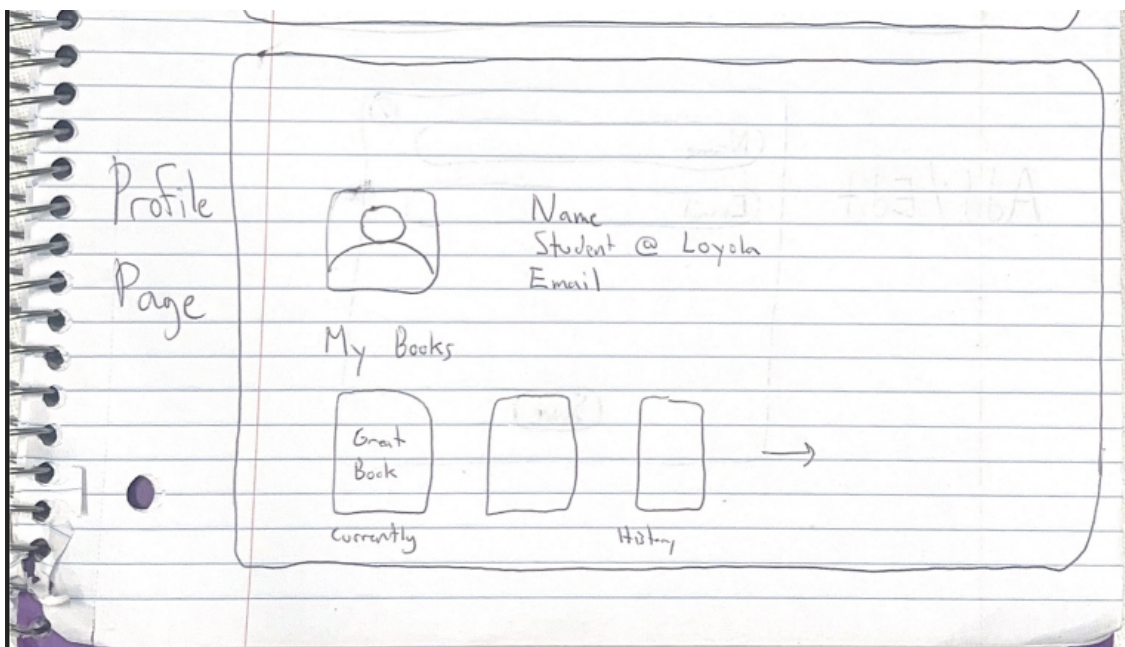


Figure 5: Profile Page

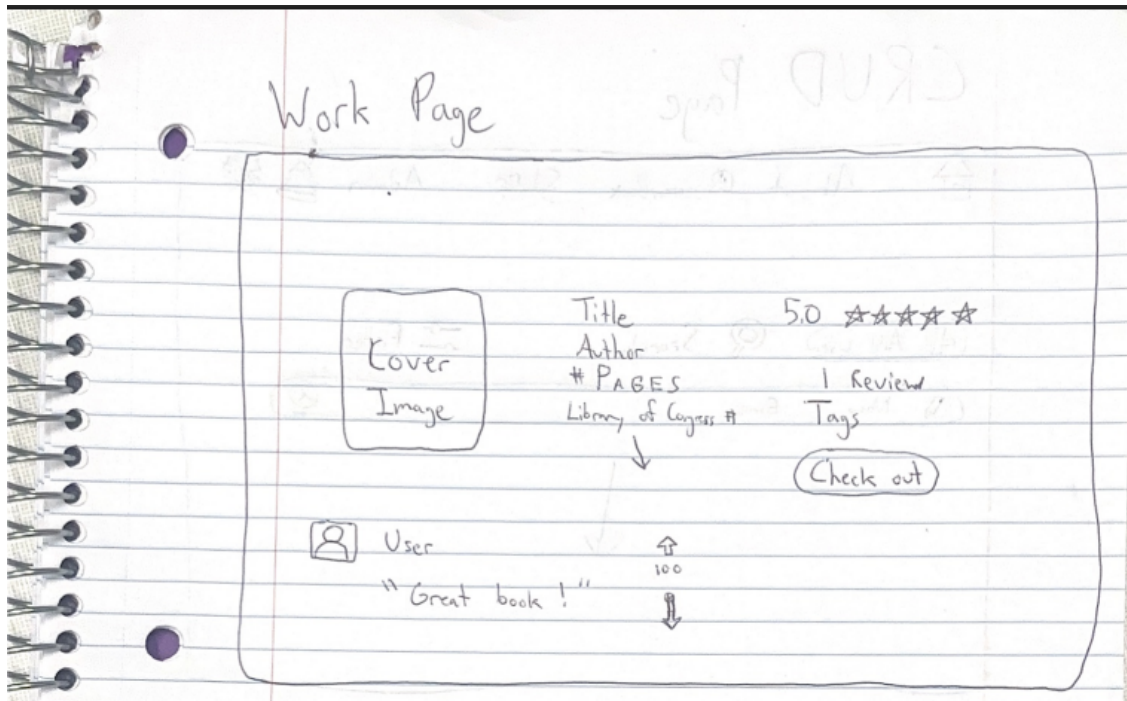


Figure 6: Work Page

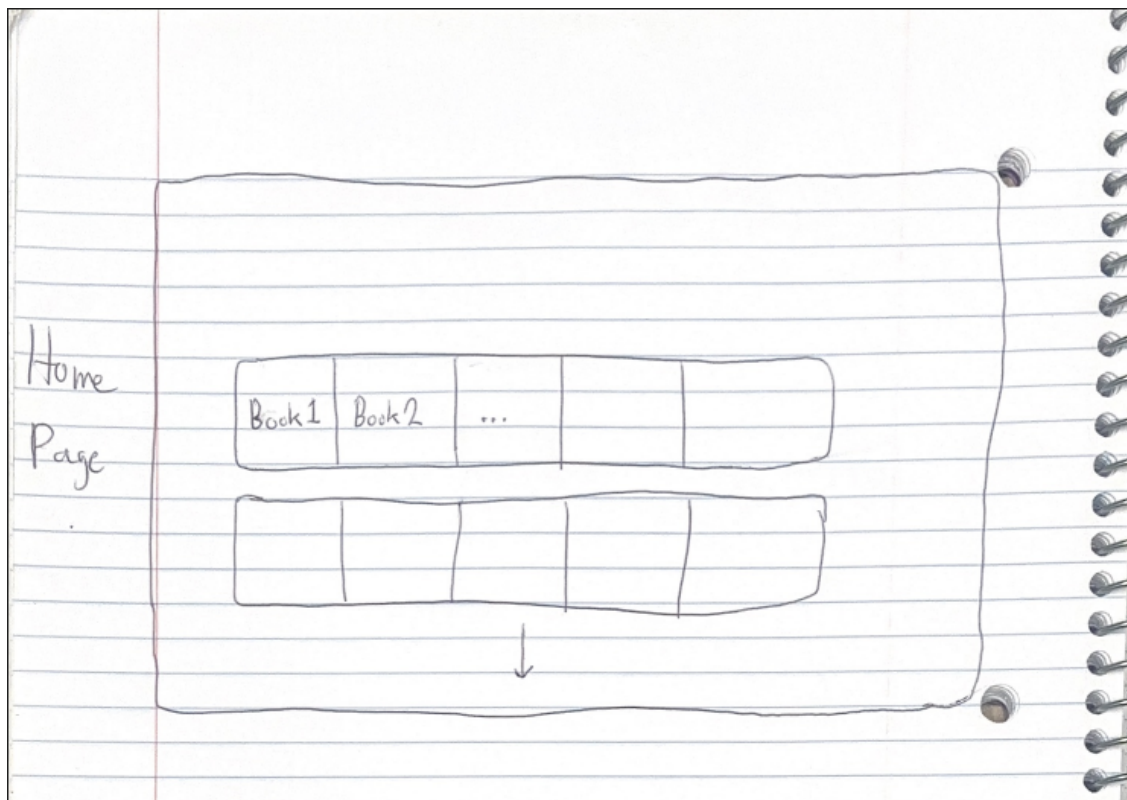


Figure 7: Home Page

5 Iterations

5.1 Iteration Planning

Table 3 shows the iteration planning.

It.	Dates	Stories	Points	
			Planned	Done
1	01/20 - 02/10	A0 Users CRUD, S1 Book CRUD, U4 Search for book	18	20
2	02/11 - 02/24	U8 View books, U26 Create account, U7 Reminder email, S11 View checked out books, U31 Log in/log out, S5 Check out/in book, S2 Fill book info via ISBN, S15 Late users, U9 Filter the view of the books, U12 Sort the view of books	17	20
3	02/25 - 03/17	S14 Tags CRUD, S3 Scan barcode, U21 Rate book, U24 Request extension, S25 Approve extension	18	
4	03/18 - 03/31	U20 View book covers, U6 Reserve book, U29 Comments CRUD, U27 User get notifications, A28 Admin send notifications, U18 Popular books, U32 Popular books by genre	19	
5	04/01 - 04/14	U10 Recommended books, U23 Book preview, U13 Upvote/downvote comments, U19 Password reset, U16 User view my history, S17 Staff view book history, U30 Report comments, U22 Contact staff	17	
Total:			89	40

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

The stories we originally planned for were A0 Users CRUD (8 points), S1 Book CRUD (8 points), U4 Search for book (2 points). We planned for a total of 18 points. The reason why we selected the Users and Book CRUDs were because they were the most important CRUDs of the app. Almost everything else in our app builds on top of these CRUDs. We also added the ability for users to search for books in iteration 1 since it is also a basic requirement that should not be too difficult after setting up a books CRUD.

5.2.2 Work Done

Aidan completed A0 Users CRUD and U31 Log in/log out for a total of 10 points. Vilnis completed S1 Book CRUD and U4 Search for book for a total of 10 points. We completed all of our stories and did not leave any partial work for next sprint. We added U31 to this sprint so that we could test the authentication of the CRUDs (although we didn't originally plan it).

5.2.3 Testing Coverage

Figure 8 shows the test coverage

We were very pleased with this testing coverage. We included testing for almost every branch in our codebase. Every file has 98% or more statement coverage. One thing our test coverage lacks is explicit front-

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.56	98.05	100	99.56	
app/api/admin/users	100	100	100	100	
route.ts	100	100	100	100	
app/api/admin/users/[id]	100	100	100	100	
route.ts	100	100	100	100	
app/api/search/works	100	100	100	100	
route.ts	100	100	100	100	
app/api/staff/works	100	95	100	100	
route.ts	100	95	100	100	46
app/api/staff/works/[id]	98.43	94.73	100	98.43	
route.ts	98.43	94.73	100	98.43	70-71
lib	100	100	100	100	
admin.ts	100	100	100	100	
staff.ts	100	100	100	100	
Test Suites: 5 passed, 5 total Tests: 52 passed, 52 total Snapshots: 0 total Time: 2.691 s Ran all test suites.					

Figure 8: Iteration 1 test coverage report

end testing. However, since we covered all the api endpoints and lib (where our authentication functions live), we are very confident in presenting this iteration.

5.2.4 Retroespective & Reflection

Aidan - A personal pitfall that I faced was using PostgreSQL. I was familiar with everything in our web stack except for this. I am more used to using NoSQL databases like MongoDB. I was excited to use PostgreSQL after finishing CS*485 Database Management Systems at Loyola. Vilnis and I spent time designing the attributes and relationships for some of our key entities. It was nice having more control over our entities.

Vilnis - First, I questioned whether I should have included author as an attribute in the Works table to make the iteration 1 product more presentable. I did not include it since we will make a separate table for Authors and have a relation between it and Works. However, if we were to present iteration 1 to our client, I wonder if they would ask where the author is displayed on the items in our Works CRUD. Second, I faced a challenge programming the search function for users. Initially, I was using something very similar to the GET work API for staff. I realized that this would not look very good because you have to wait to fully type in a search to return something, and if it is not exact, you would get nothing. Therefore, I implemented a search feature that actively updates as the user types in the search bar using SQL ILIKE. I was happy with this solution.

5.3 Iteration/Sprint 2

5.3.1 Planning

For this iteration, we planned to complete U8 View books (2 points), U26 Create account (2 points), U7 Reminder email (2 points), S11 View checked out books (2 points), U31 Log in/log out (2 points), S5 Check out/in book (1 point), S2 Fill book info via ISBN (2 points), S15 Late users (2 points), U9 Filter the view of the books (1 point), U12 Sort the view of books (1 point). This added up to a total of 17 points.

After completing the most important CRUDs in Iteration 1, we thought the best next thing to do was complete the highest priority features. Every user story in this iteration was labeled as high or medium priority. There were no large features planned for this iteration, and the largest user story we planned had 2 story points.

5.3.2 Work Done

Our iteration plan changed due to a couple of factors. First, U31 Log in/log out was completed in Iteration 1, so it was removed from Iteration 2. We decided to add U8 View books, because it was the same functionality as the read operation in S1 Book CRUD. We modified Check out/check in book from 1 story point to 5 story points because we underestimated how difficult it was going to be. We had to create a new checkout entity. We decided to add S3 Scan barcode, U24 Request Extension and S25 Approve Extension to this iteration as well. We changed the amount of points for S3 Scan barcode from 5 to 1 because we learned that you can plug a barcode scanner into an HDMI port, and it will fill out the input field when a barcode is scanned. All of these stories brought us to a total of 20 points for this iteration.

Aidan completed S5 Check out/in book, S11 View checked out books, U26 Create account, and S3 Scan barcode for a total of 10 points. Vilnis completed U7 Reminder email, S2 Fill book via ISBN, S15 Late users, U9 Filter the view of books, U24 Request extension, and S25 Approve extension for a total of 10 points. We completed all 20 points that we mapped out in our adjusted plan.

5.3.3 Testing Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.15	98.04	98.55	99.15	
app/api/admin/users	95.83	94.73	100	95.83	
route.ts	95.83	94.73	100	95.83	14-15
app/api/admin/users/[id]	97.05	90	100	97.05	
route.ts	97.05	90	100	97.05	23-24
app/api/auth/register	100	100	100	100	
route.ts	100	100	100	100	
app/api/search/works	100	100	100	100	
route.ts	100	100	100	100	
app/api/staff/checkouts	100	100	100	100	
route.ts	100	100	100	100	
app/api/staff/checkouts/[id]	100	100	100	100	
route.ts	100	100	100	100	
app/api/staff/works	100	100	100	100	
route.ts	100	100	100	100	
app/api/staff/works/[id]	97.36	89.47	100	97.36	
route.ts	97.36	89.47	100	97.36	16-17
app/api/staff/works/lookup	100	100	100	100	
100 33					
isbn-lookup.ts	100	100	100	100	
staff.ts	100	100	100	100	
validation.ts	100	100	100	100	
lib/data	98.98	98.7	96.96	98.98	
checkouts.ts	97.14	94.87	92.3	97.14	118-126
sessions.ts	100	100	100	100	
users.ts	100	100	100	100	
works.ts	100	100	100	100	
Test Suites: 15 passed, 15 total Tests: 282 passed, 282 total Snapshots: 0 total Time: 4.487 s Ran all test suites.					

Figure 9: Iteration 2 test coverage report

We are very pleased with our testing coverage in this iteration as well. The uncovered lines correspond to rare edge cases and error-handling branches that are difficult to trigger in a test environment. As with Iteration 1, our test coverage still lacks explicit front-end testing. However, given that we have near-complete coverage of all API endpoints and our data library, we are very confident in the correctness of our backend. One notable gap is that we have not yet written automated tests for our email reminder service, as it relies on an external provider (Resend) that is difficult to test reliably without mocking. In Iteration 3, we plan to address this by mocking the email client in our test environment, and we will continue to maintain this high standard of coverage. We will also need to re-evaluate the importance of front-end testing as the project grows. We may implement snapshot testing or React Testing Library tests. This will help us feel secure in shipping the final product live.

5.3.4 Retroespective & Reflection

Aidan - One challenge that we ran into this iteration was having to modify our plan. For the first iteration, we stuck to our plan nearly perfectly. This sprint was completely different because of the amount of changes we made (see section 5.3.2). Something we could have done differently was to spend more time planning all of our iterations before we began iteration 1. However, it was hard to foresee some of the changes that we had to make. Future iterations will most likely have the same problem because we already pulled tickets from future sprints into Iteration 2. We will live with the changes that we have already made to future iterations, and we will try our best to make the least amount of changes as possible.

Vilnis - Although we will have challenges foreseeing which tasks will become the priorities in subsequent iterations, this is the advantage of Agile methodology. We have the flexibility to adapt to changing requirements. One big endeavor that was missed in the user stories was my refactoring of the API layer and addition of Data Access Layer that contains all the raw SQL. This will help us reuse data access functions and maintain a clear separation of concerns. In Iteration 2, my biggest challenge was designing, implementing, and testing the email reminder feature. It took time to research which email provider to use (I settled on Resend) and set up an API key. Furthermore, manually testing the service is not reliable and we need to implement mocks in Iteration 3. Last, the email that a user receives comes from a weird address and was sent straight to my Junk folder. Overall, we may need to completely refactor the email reminder feature. However, it technically satisfies the user story, so I counted it as done.

5.4 Iteration/Sprint 3

5.4.1 Planning

5.4.2 Work Done

5.4.3 Testing Coverage

5.4.4 Retroespective & Reflection

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482)]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid

the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroerspective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]